

Exercise 10

File I/O and Overflow

Up to this point, our programs have been reading input and writing output on standard in and standard out. For this program, you will be using the standard library to open and read input from a file. You'll also be using C standard library functions to help parse the input, and you'll be writing a little bit of code to help detect overflow.

This exercise has a starter source file and some sample inputs and expected outputs. You can download these from the course homepage, or, if you're working on one of the NCSU remote Linux machines, you can copy them all to your current directory with the following command:

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise10/* .
```

You'll be completing a program named **calc.c**. This program reads statements from an input file named **script.txt**. As it executes statements from the input file, it will print results to the terminal. If errors are encountered while executing the statements, it will print error messages to standard error.

Variables and Statements

The **calc.c** program is like a little calculator. It lets the user store values in 26 different variables, named **a .. z**. The values of these variables can be used or modified by statements in the **script.txt** input file. For example, the first test input file contains the following:

```
a=7
b=-4
c=a+b
print c
```

This script sets variable **a** to 7, then it sets **b** to -4. It adds the values of **a** and **b** and stores the result in variable **c**. Then, the last statement prints the value of **c** to the terminal.

Variable values and arithmetic operations on them are performed using the (signed) long type. The starter code already has a global variable for storing the current value of each variable, and two functions, **get()** and **set()**, for getting and modifying the value of one of the variables.

The **execute()** function in **calc.c** is responsible for reading and executing the next statement in the **script.txt** input file. It returns true if it is able to parse another statement from the input. It returns false if it encounters an end-of-file or a syntax error (i.e., a statement that doesn't match one of the statement types below).

The starter just has code to handle just one type of statements. To complete your implementation, you will need to add code to handle the remaining statement types shown below. This will be a good chance to practice using the pattern matching syntax supported by **scanf()/fscanf()**. It's easy to write a little conversion specifier to match a variable name in the input, and remember that most literal characters in a format string will match a copy of themselves in the input. You should be able to write a little format string to match each type of expression below.

```
• a = b + c
  a = b - c
  a = b * c
  a = b / c
```

An input statement like one of these will add, subtract, multiply or divide the values of two variables and store the result in another variable. The examples above show what these would look like for

variables a, b and c, but statements like these could use any lower-case letters as variable names. These statements only perform math on values stored in variables. You can't directly use a numeric value; you have to put it in a variable first. This should make it easier to parse the input. You don't need to be able to handle statements like the following:

```
x = y + 1
z = 100 * 3
```

The starter already has code to handle the first type of statement, including code for detecting overflow.

- `a = 1234567890`

An assignment statement like this can be used to store a long value in a variable. Of course, the variable on the left could be any lower-case letter and the value assigned to it can be any signed long value.

- `print a`

The print statement prints out a line to the terminal reporting the current value given variable. We're showing an example for variable, a, but any lower-case letter could be used as the variable name.

Input statements could contain additional space or no space between input tokens. The `fscanf()` function makes it easy to skip arbitrary amounts of whitespace, so permitting any amount of space should make the parsing easy.

File Input

You will need to add code to open a file named "script.txt" at the start of program execution and to close it before exiting. Use the variable, `fp`, to store the FILE pointer for this stream. The starter already has code to pass this pointer to the `execute()` function each time we try to parse a statement.

Be sure to check to make sure that the input file was successfully opened. If not, print a line with the following error message to standard error and terminate the program with an exit status of 1.

```
Can't open script.txt
```

Statement Parsing

The starter just has code to parse one type of expression using `fscanf()`. You will need to add code to parse all the other types of statements supported by our program. For each type of statement, you should be able to write a little format string to get `fscanf()` to parse the statement. The return value of `fscanf()` to tell if it successfully parsed the type of statement it was trying to match.

Of course, even if `fscanf()` can't parse the statement, it could still consume some characters from the input, if, for example, it matches some characters at the start of the format string but can't match the whole thing. If we want to try parsing the input as a different type of statement, we'll need to back up in the input so we can try to parse again from the same starting point. This is a good job for `ftell()` and `fseek()`.

Add a call to `ftell()` at the start of `execute()` to remember the location in the input file when we started trying to parse the current statement. Then, after a call to `fscanf()` fails to match a particular type of statement, you can use `fseek()` to go back to that location before trying to parse the input with a different format string.

Overflow Detection

For any of the arithmetic operations, it's possible for an overflow to occur. If this happens, the `calc.c` program should ignore the statement and print a line like the following to standard error. Here, the variables reported

should be the ones used in the statement that caused the overflow, and the operator should be whatever operator was used in the statement. After reporting this error message, the **calc.c** program should continue trying to read and execute subsequent statements from the **script.txt** input file.

Overflow: b + c

For detecting overflow, the starter contains four functions, one for each of the four arithmetic operations. For example, **checkAdd()** takes two long integer parameters. It returns false if adding these two values results in an overflow. You'll be using four functions like this to detect when an add, subtract, multiply or divide operation overflows. The **checkAdd()** and **checkDiv()** functions are already implemented for you, and **checkMul()** is partially implemented. You'll need to fill in the implementation of **checkSub()** and complete the implementation of **checkMul()**.

The **checkAdd()** function detects overflow in addition operations by considering two cases where overflow could occur. Adding two non-negative numbers should give a non-negative result. It results in a negative number, then an overflow has occurred. Likewise, adding two negative numbers should give a negative result. If the sign of the result is wrong, then an overflow has occurred. You should implement **checkSub()** similarly. Based on the signs of the two numbers, there are two cases where an overflow could occur during a subtraction. For example, subtracting a non-negative number from a negative number should give you a negative result. If the result is positive, then an overflow has occurred. In your implementation of **checkSub()**, fill in code to recognize the two cases when an overflow could occur, and use the sign of the result to tell whether the given values actually cause an overflow.

Catching overflows in multiplication is a little more interesting. We can't just use the sign of the result like we can with add and subtract. For multiplying values **b** and **c**, you will need to write a little bit of code for each of four different cases, depending on the signs of **b** and **c**. We'll also need the constants **LONG_MAX** and **LONG_MIN** so we can consider the largest of smallest (most negative) value that fit in a long integer.

The starter already has code for the case when **b** and **c** are both positive. In this case the result should be no larger than **LONG_MAX**, but we can't just compute **b * c** and compare that with **LONG_MAX** (no value stored in a long could ever be larger than **LONG_MAX**). Instead, for a given value of **b**, we can figure out the largest value **x** such that **b * x** is less than or equal **LONG_MAX**. We just have to compute **x = LONG_MAX / b**. When computing **x = LONG_MAX / b**, integer division will discard any remainder, rounding down (toward zero). So, with **x** computed like this, **b * x** could be smaller than **LONG_MAX**. If **LONG_MAX** is evenly divisible by **b**, then **b * x** will be exactly equal to **LONG_MAX**. Either way, this **x** value is the largest value that we can multiply by **b** without exceeding **LONG_MAX**. So, if **c** is larger than **x** then **b * c** will overflow.

For example, if **b** is **1234567890**, then **x = LONG_MAX / b** gives us **7470931417**. That's the largest value we can multiply by **b** without getting a signed, 64-bit overflow. You can check, **1234567890 * 7470931417 = 9223372035820400130**, which fits in a long int on the common platform. If we try a slightly larger value for **c**, the correct result would be **1234567890 * 7470931418 = 9223372037054968020**, but we won't get this if we're computing with the long type on common platform. The result of **9223372037054968020** is too big to fit into a long.

The existing code takes care of the case where **b** and **c** are both positive. You'll need to figure out similar tests for the other three cases, where **b** is positive and **c** is negative, where **b** is negative and **c** is positive or when both values are negative. Each of these cases will be similar, and they can each be handled with about three lines of code.

Divide by Zero Detection

For the division operation, you will also need to add a check for divide-by-zero. If the script attempts to divide by zero, print a line with the following message to standard error, ignore the statement with the divide-by-zero error, but continue executing subsequent statements in the **script.txt** file. In this message, the **b** and **c** should be the actual variables used as operands to the division operation.

```
Divide by zero: b / c
```

Syntax Errors

If the `script.txt` input file contains text that doesn't match any of the six types of statements, the program should print a line with the following message to standard error, then terminate with an exit status of 1. The starter already contains code to handle this at the end of the `execute()` function.

Running the Program

Once you have a working program, you should be able to test it using commands like the following. The program always reads from an input file named `script.txt`. The starter includes four test input files named `script-1.txt`, `script-2.txt`, `script-3.txt` and `script-4.txt`, but you will have to copy these files to a file named `script.txt` to use it as input for your program. The following shows the commands you can run to test your program with the input file, `script-1.txt`. First, we copy it to `script.txt`. Then, we run the `calc.c` program with output sent to the file `output.txt`, but without any input redirection. The program should automatically read input from the `script.txt` file. After running the program, we check to make sure it exited successfully, and then we check to make sure the output file it created looks right.

```
$ cp script-1.txt script.txt
$ ./calc > output.txt
$ echo $?
0
$ diff output.txt expected-1.txt
```

The `script-1.txt` input file just sets a couple of variables, adds them and prints out the result. The second test input file includes more whitespace in the input and uses subtraction, multiplication and division operations. You should be able to check your program against this test case using command like the following:

```
$ cp script-2.txt script.txt
$ ./calc > output.txt
$ echo $?
0
$ diff output.txt expected-2.txt
```

The `script-3.txt` file includes lots of statements that result in the largest or smallest value that can be stored in a signed long. It also includes tests that should be flagged as an overflow or a divide by zero. For example, the file starts out with the following statements:

```
a = 9223372035620207917
b = 1234567890
c = 1234567891

d = a + b
print d

e = a + c
print e
```

The sum of the `a` and `b` variables gives the maximum value that a long can store. The variable, `c`, is one larger than `b`, so `a + c` causes an overflow, and the variable, `e`, isn't modified. This first part of the file should generate the following output:

```
9223372036854775807
```

```
Overflow: a + c
0
... more output ...
```

The following commands will try out your code with test script 3. Notice that, when we run the `calc` program, we're redirecting standard output to one file and standard error to a different file. Then, we check both of these files to make sure they contain the expected output. If you let your program's output go to the terminal, output to both of these streams will show up together. Sending them to different output files lets us tell what output went to each stream.

```
$ cp script-3.txt script.txt
$ ./calc > output.txt 2> stderr.txt
$ echo $?
0
$ diff output.txt expected-3.txt
$ diff stderr.txt expected-stderr-3.txt
```

The last test input checks to make sure the program terminates when it hits an input line that it can't parse. It should be easy to pass this test. The starter code given with this exercise should already pass it.

```
$ cp script-4.txt script.txt
$ ./calc 2> stderr.txt
$ echo $?
1
$ diff stderr.txt expected-stderr-4.txt
```

Submitting your Work

This is the first program we've written that opens a file and read input from it. Up to this point, we've mostly used input redirection to get our program to read input from a file. Be sure your program works as demonstrated above. It shouldn't expect the user to run it with `<` on the command line to redirect standard input from a file.

When you're done with your program, submit your completed **calc.c** source file via the `exercise_10` assignment on Moodle.